# Terminator on SkyNet: A Practical DVFS Attack on DNN Hardware IP for UAV Object Detection

Junge Xu[◇•], Bohan Xuan[•○], Anlin Liu[•], Mo Sun[•], Fan Zhang[✉◇★], Zeke Wang[◇] and Kui Ren[◇]
College of Computer Science and Technology, Zhejiang University[◇]
Alibaba-Zhejiang University Joint Research Institute of Frontier Technologies[★]
Key Laboratory of Blockchain and Cyberspace Governance of Zhejiang Province[○]
College of Information Science and Electronic Engineering, Zhejiang University[•]

## ABSTRACT

With increasing computation of various applications, dynamic voltage and frequency scaling (DVFS) is gradually deployed on FPGAs. However, its reliability and security haven't been sufficiently evaluated. In this paper, we present a practical DVFS fault attack targeting at the SkyNet accelerator IP and successfully destroy the detection accuracy. With no knowledge about the internal accelerator structure, our attack can achieve more than 98% detection accuracy loss under ten vulnerable operating point pairs (OPPs). Meanwhile, we explore the local injection with 1 ms duration and next double the intensity which can achieve more than 50% and 74% average accuracy loss respectively.

## 1 INTRODUCTION

Given their high flexibility and energy efficiency, FPGAs have become one of the most promising edge computing platforms and have been widely deployed as hardware accelerators for various neural networks. Therefore, neural network implementation on FPGA has drawn close attentions in recent years. Some researches have shown that FPGA-based hardware accelerator can be a superior solution in terms of both throughput and energy efficiency when convolutional neural networks (CNN) are trained in particular circumstances [10]. Although the implementation of neural networks is achieving better performance, the raising concerns on its robustness and security have not been adequately summarized and fully addressed.

The need for better performance, power- and energy-efficiency drives the development of modern hardware-software energy management technology. DVFS mechanism is one of the most commonly used technologies. In order to maximize the utility of it, hardware and software operate cooperatively at very fine granularities to regulate the frequency and voltage of processors. The author [8] combined the fine-grained DVFS framework with CNN-based object detection accelerator in FPGA . However, considering the complexity of software-hardware interoperability requirements and the pressure of cost and time limit to the market, designers of DVFS may do not fully consider the security of it, which makes it possible to break through the safe defense-line of DVFS with only software methods.

Fault injection attack is an important kind of method to destroy the accuracy of neural networks through modifying electronic circuits' behaviors. Various fault attacks on FPGAs such as glitch disturbance [1], rowhammering [9] and glitch injection [11] have been demonstrated previously. Most of recent fault attacks targeting at neural networks focus on model weight manipulations [6], which will leave traces in memory no matter how much data has been manipulated. Such persistent fault injection into the networks can not avoid detection and can be bypassed by parameter reloading. Inconspicuous fault attack targeting at neural networks' accelerators has been payed close attentions on recently and has been proposed in several papers, such as glitch injection [11]. However, there is no specific work on DVFS fault attacks on FPGAs.

In this paper, we summarize the research gap in related fields and proposed the first stealthy and non-invasive fault attack targeting at neural network hardware IP based on DVFS mechanism. In specific work, we combine the DVFS framework with the SkyNet accelerator. And the DVFS framework consists of DVS and DFS modules implemented by some hardware components which mapped to corresponding FPGA drivers. Therefore, hardware fault can be injected through software interfaces to corresponding FPGA drivers. Through these software interfaces, we can dynamically control the voltage and frequency out of specification and thus induce abnormal behaviors of electronic circuits. When the voltage/frequency pair is in different value areas of the two-dimensional coordinates, the induced fault will cause three different results: silent data corruptions (SDCs), accelerator hangs, or system crashes. SDCs are the basic of our inconspicuous attack while an accelerator hang or system crash will result in function disruption and arouse vigilance. Our attack, like a terminator on the detection accuracy of SkyNet, aims at inducing temporal faults into the intermediate results of the network which will propagate to the inference stage and lead to accuracy loss.

The main contributions of our work are summarized as follows:

• We present the first DVFS fault attack on FPGA, which is stealthy and does not require any physical access to hardware.

• We design the architecture of our evaluation system with the MMCM module and the PMBus compliant power supply system which consists of a DVFS framework and the SkyNet accelerator. And we show how to induce SDC fault into the accelerator.

• We evaluate our attack on the DAC-SDC2019 champion SkyNet which aims at real-world UVA single object detection. And we investigate both global and local fault injection. For global fault injection, our attack can achieve more than 98% detection accuracy loss. For local fault injection, we can achieve more than 74% average accuracy loss with 95.45% fault probabilities.

The remaining part of this paper will be organized as follows. In Section 2, we talk about some previous related works on DVFS methodology, neural network accelerator and fault attack. In Section 3, we describe our threat model, the violation of timing constraint

*DAC '22, July 10–14, 2022, San Francisco, CA, USA*

ACM ISBN 978-1-4503-9142-9/22/07…$15.00
https://doi.org/10.1145/3489517.3530516

and our DVFS framework. In Section 4, we explore the characterization of regulator limits and two black-box attacks. In Section 5, we make our final conclusion.

# 2 RELATED WORKS

## 2.1 DVFS Methodology

DVFS has been proven to be an efficient system-level methodology and has been widely deployed on ASICs, CPUs and GPUs. For energy efficiency optimization of neural network accelerators, there have been some previous works. The authors of [13] developed a principled approach and a data-driven analytical model to optimize granularity of threads during convolutional neural network (CNN) software synthesis. The experimental results with several modern CNNs mapped to an Android phone with a Snapdragon SoC showed up 2.37X speedup in application runtime and up to 1.9X improvement in its energy dissipation compared to existing approaches. The authors of [2] proposed a low-power CNN-based face recognition system for user authentication in smart devices with DVFS. The authors of [16] used a performance-power analytical model fitted on a parametrized implementation of a deep learning accelerator in a 28-nm FDSOI technology to explore a large design space and to obtain the Pareto points that maximize the effectiveness of DVFS in the sub-space of throughput and energy efficiency. On FPGAs, The authors of [8] first combined CNN-based object detection with DVFS on FPGA and achieved 38% improvement in energy efficiency without any loss in accuracy.

## 2.2 Neural Network Accelerator

Previous works have shown that neural networks have significant advantages in machine learning over traditional algorithms. However, their applications on energy- and resource-constrained edge devices are limited by the high demand on computation capability, memory resources. In recent years, efficient hardware accelerators have been developed to enable their applications. Due to a coarsely quantized numerical representation reducing memory requirement and saving power consumption, data quantization is the primary technique used to accelerate neural networks [4] [5].

In our work, we evaluate our proposed attack on Skynet [17] which won the championship in Design and Automation Conference-System Design Contest 2019 (DAC-SDC2019), a low power object detection challenge in images captured by unmanned aerial vehicles (UAVs). SkyNet has 40 layers totally, 12 of which are convolutional (Conv) layers and delivers 0.716 intersection over union (IoU) and 25.05 FPS on an Ultra96 FPGA. One of the reasons why we choose SkyNet as our test model is that it represents the highest level of deep neural network (DNN) implementation on FPGAs in terms of both performance and energy efficiency. The other reason is that it is a highly encapsulated application on FPGA which does not reveal any internal data or construction of the model. According to such characteristics, we can consider SkyNet as a highly encapsulated real-life commercial application in the UAV detection scenario, which makes our attack have more realistic significance.

## 2.3 Fault Injection Attack

Fault injection attack interferes with the behavior of underlying hardware and finally causes erroneous output at the software level. Despite the great achievements of DNNs along with their implementations on FPGAs, the vulnerability of DNN hardware has raised high security concerns and has been revealed in several works of fault injection attack. Work [18] presented a fault sneaking attack which tampers model parameters and misclassifies selective images. However, the misclassification process assumed that any parameter can be modified to an arbitrary value. The authors of [7] proposed the rowhammering attack to manipulate model weights in high performance computing cluster. They also suggested that such attack can be mitigated with low-precision numeral representation. Physical fault injection attack on DNNs was studied in [3] which uses laser beam to disturb hardware operations targeting at activation functions and finally misclassifies input images. As for the fault injection attacks on FPGAs, an imperceptible misclassification attack was presented in [12], which perturbs the clock signal of the PE array infrequently to disturb DNN outputs. Their exploratory study focuses on FPGA-based DNN accelerators in both black-box and gray-box attack scenarios. For the black-box attack, above 98% misclassification on 8 out of 9 ImageNet models can be achieved with 10% glitch intensity. For the gray-box attack, only 1.7% glitches are required to be injected to successfully subvert the correct classification results of all the images input to the hardware accelerator of ResNet50.

DVFS-based hardware fault attacks in several hardware platforms have been attempted recently. On Intel CPU, researchers proposed SVHF [14], a software-controlled voltage-based hardware fault attack for software guard extensions (SGX). SGX allows applications to run in a fully trusted area, which provides a high degree of security for applications. SVHF uses the characteristics of DVFS technology to verify that the hardware fault can be injected into the victim program and lead to differential output by configuring the voltage and frequency of Intel processor. Experiments show that the attack method can successfully obtain the encryption key of AES protected by SGX. On the AMD GPU, researchers proposed an overdrive fault attack targeting at AMD GPUs [15]. Similarly, using the voltage-frequency scaling function owned by most processors, random faults can be introduced during kernel execution. The experimenters realized an effective, non-invasive and hardware fault based attack against modern GPU voltage-frequency scaling system on AMD GPU platform. In the experiment, they implemented this fault injection attack on an AES ECB encryption kernel and successfully extracted all the 16 key bytes.

# 3 ATTACK DESIGN

In this section, we first describe our threat model and the intention of our attack. Next, we demonstrate how the SDC fault occurs when the frequency/voltage pair stretch beyond the operating limit of digital circuits. Finally, we describe our dynamic voltage and frequency scaling framework and how it regulates the frequency/voltage pair of the SkyNet accelerator.

## 3.1 Threat Model

Our practical DVFS attack aims at destroying the accuracy of neural network accelerator IP in real-world scenarios, such as SkyNet demonstrated before. For highly encapsulated accelerator IP, we assume that the attacker only knows the general architecture of the

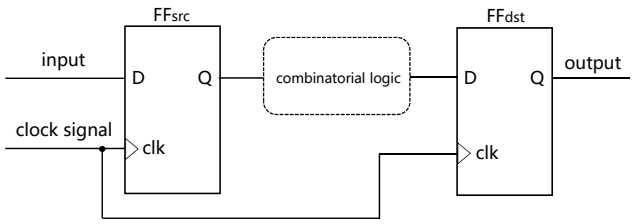


**Figure 1: Flip-flops**

hardware IP but not the low level implementation details, such as the hardware construction of each layer. Based on this premise, in general, we take two kinds of black-box attack modes into consideration, the global fault injection and the local fault injection. For global fault injection, we induce fault into the whole procedure of SkyNet inference. According to different regulation of the frequency/voltage pair, different degrees of the object detection accuracy corruption will occur, regardless of the input images. However, due to the consideration of the attack cost and stealthiness, we set our sights on a shorter fault injection time. For local fault injection, we induce fault with a certain duration and set different injection starting time in each round of experiments. At the meantime, we roughly estimate each layer's runtime of SkyNet accelerator IP in accordance with the FLOPs (floating point operations) of the SkyNet model on GPU. So we can infer the exact injected layer according to the injection starting time to some extent. It is worth noting that such infer may not be totally accurate, since DVFS attack will perturbate the execution time of the hardware.

## 3.2 Violation of Timing Constraint

In most FPGA digital circuit designs, flip-flop (FF) is one of the most commonly used components which preserves the stateful result of computations. Flip-flops transfer data from input port to output port only when the rising edge of the clock signal arrives. In Fig. 1, we show some parts of circuits containing FFs that share same synchronous clock and some combination logic elements between them. The correct data propagation needs to meet the flip-flops timing constraint as follows:

$$T_{clk} \geq T_{setup} + T_{FF} + T_{propagation} \tag{1}$$

where $T_{clk}$ is the period of clock signal, $T_{setup}$ is the holding time of input data before the rising edge of clock signal, $T_{FF}$ is the propagation delay from input to output in flip-flops, $T_{propagation}$ is the propagation delay of combination logic between two FFs.

The timing constraint can be violated through overclocking to shorter $T_{clk}$ or undervolting to longer $T_{propagation}$ and $T_{FF}$ due to the lower supply voltage. When the timing constraint is violated and the output of $FF_{dst}$ is flipped, the SDCs fault will occur. According to DVFS mechanism, we dynamically adjust the frequency/voltage pair to violate the timing constraint and thus induce random faults to the SkyNet accelerator.

## 3.3 Dynamic Voltage and Frequency Scaling Framework

Our DVFS framework can provide dynamic frequency/voltage regulation with high resolution, low scaling time and wide range. In practical regulation, it can adjust the voltage in the range of 550mV to 1100mV at a step size of 10mV in 2 ms approximately and adjust the frequency in the range of 20 to 500 MHz at a step size of 5MHz in 3 $\mu$s. The DVS module is implemented with the power management IC (PMIC) through controlling and monitoring the switching regulators which provide different voltage to various components on FPGA, such as the SkyNet accelerator. It can be accessed by the processing system (PS) via I2C interface. The DFS module is implemented with the Mixed-Mode Clock Manager (MMCM), which can generate all the desired clock signals of the SkyNet accelerators on the programmable logic (PL). The PS controls the target frequency of DFS module via AXI interface provided by Zynq platform.

**1) Dynamic Frequency Scaling:** Due to the requirement of high resolution and wide scaling range, we choose MMCM to implement our DFS module for generating target frequency. In MMCM, the target frequency can be calculated by the following equation:

$$F_{OUT} = F_{CLKIN} \times \frac{M}{D \times O} \tag{2}$$

The parameter M, D, and O can be configured through dynamic scaling port in the MMCM and thus the target frequency can be generated at run-time.

As shown in Fig. 2, the configuration information and the reset signal are sent to the register map via the AXI4-Lite interface. When the MMCM is ready, it will obtain the information saved in the register map and generate the target frequency accordingly.

**2) Dynamic Voltage Scaling:** The-state-of-the-art FPGA boards usually adopt the power management IC and a PMBus-compliant system controller to supply core and auxiliary voltages. The Power Management Integrated Circuit (PMIC) of our test board Ultra96-V2 is Infineon IRPS5401, which follows the power management bus (PMBus) protocol. The PMBus protocol is extended based on the I2C protocol and is compatible with it at the physical level.

As Fig. 3 shows, the PMIC obtains the configuration information from the FPGA through the I2C bus and transmits multiple voltages generated in different switching regulators back to the FPGA. In the meantime, the PMIC monitors and reports the statuses of each switching regulator back to the FPGA and thus we can observe the exact value of the set voltage. We can leverage the integrated I2C driver in Linux to control the PMIC to generate the target voltage.

# 4 EVALUATION RESULTS

First, we introduce the experiment setup when we implement our DVFS framework. Next, we characterize the regulator limits of the target accelerator. Finally, for global fault injection, we explore these vulnerable OPPs leading to different degrees of accuracy corruption. And for local fault injection, evaluations are carried out to find out the impact of different fault injection durations and fault injection points.

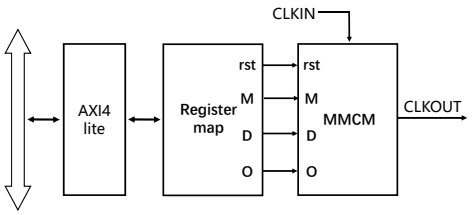


**Figure 2: MMCM Configuration Module**

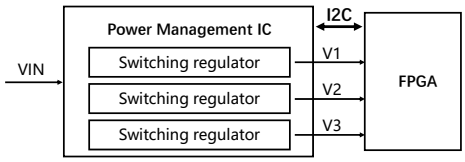


**Figure 3: PMIC Framework on FPGA**

## 4.1 Experimental Setup

The SkyNet accelerator and the DVFS framework support a series of Zynq 7000 and Zynq UltraScale+ series FPGA boards. We configure our experimental setup as shown in Fig. 4. It is implemented on Xilinx Ultra96-V2 evaluation board which features a ZYNQ UltraScale+ (XCZU3EG-SBVA484) MPSoC device and integrates Zynq processing system (PS) and programmable logic (PL).

Our setup consists of a SkyNet accelerator, DVS module, DFS module, a host PC, an ARM processor and some peripheral circuits. The programming language used in the SkyNet accelerator is high-level synthesis (HLS) C++ and the related codes are synthesized in SDSoC 2019.1. The DVS module is implemented with PMIC and can communicate with the PS via MIO. The DFS module provides all the clock signals of the SkyNet accelerator. The implementation details of the DVS and the DFS module are introduced in Sec.3.3. The test dataset prepared for our attack evaluations is the public dataset of DAC-SDC2019 which is composed of 1000 images and has corresponding ground truth labels.

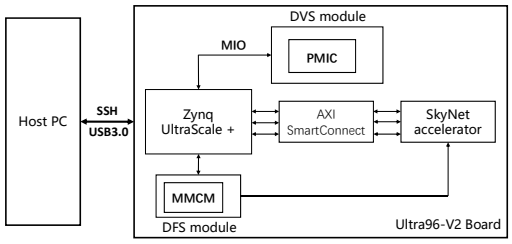


**Figure 4: Block Diagram of Experiment Setup.**

## 4.2 Characterization of Regulator Limits

In this section, we characterize the hang-crash boundary of the accelerator within which the accelerator IP can operate normally without a crash or hang. Under our DVFS regulation, the SkyNet accelerator works under a series of designated frequency/voltage pairs which are called operating point pairs (OPPs). At different OPPs, the performance and detection accuracy of the SkyNet accelerator is quite different, especially when OPPs approach the hang-crash boundary which we will demonstrate later.

We scan the OPPs by incrementally increase the voltage from about 580 mV to 1100 mV at a certain frequency between 250 MHz to 480 MHz to find the minimum voltage below which the hardware IP would hang or crash. For the concern of the scaling resolution,

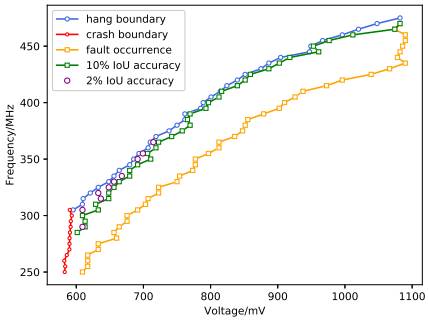


**Figure 5: The Manipulable OPPs for Ultra96-V2 Loaded with SkyNet**

the scan step is 5MHz for frequency and 10mV for voltage. As the result shown in Fig. 5, when the frequency is higher than 305 MHz and the voltage is on the left of the blue curve, the SkyNet accelerator will hang. When the frequency is lower than 305 MHz and the voltage is on the left of the red curve, the SkyNet accelerator will crash before hang due to the low supplied voltage. The red and blue curves together are called hang-crash boundary. According to the boundary curve , we are able to find out those OPPs under which we can continue the subsequent experiments without a hang or a crash. More than half OPPs in our scalable range meet such requirements. And we assume that when the accelerator works under those OPPs that near the boundary (but doesn't hang or crash), the detection accuracy will be corrupted with high probabilities.

## 4.3 Global Fault Injection

Under the vendor-stipulated OPP, the detection result of a certain image is always the same in different runs with no SDCs and thus we name these result *the normal truth*. For better evaluations of the accuracy corruption under vulnerable OPPs, we choose *the normal truth* as the baseline to calculate the IoU in fault quantification. The IoU can be calculated by the following equation:

$$IoU = \frac{DetectionBoundingBox \bigcap NormalBoundingBox}{DetectionBoundingBox \bigcup NormalBoundingBox} \quad (3)$$

where the detection bounding box is the prediction result under a selected OPP and the Normal Bounding Box is *the normal truth*. Lower IoU represents more corrupted prediction result, hence represents a stronger attack. Therefore, we calculate the average value of

**Figure 6: Object Tracking Results Generated by SkyNet at 350 MHz and 692 mV.**

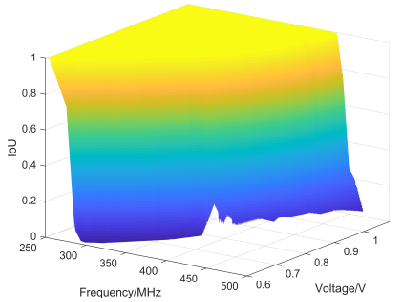


**Figure 7: IoU Change with Voltage and Frequency Respectively.**

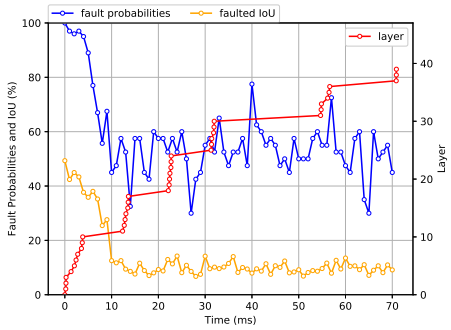


**Figure 8: Layer Injection Result**

the IoU over the test dataset under each OPP in our scalable range. The result is shown in Fig. 7. For every frequency from 250 MHz to 480 MHz, there is always a voltage threshold below which the average IoU will drop from 1. We gather these OPPs and represent them with the orange curve in Fig. 5. It can be observed that there is an immediate fall for the average IoU when OPPs between the hang-crash boundary and the fault occurrence boundary. Through multiple evaluations on the dataset, we explore the ten most vulnerable OPPs marked as pink dots where the average IoU can decrease below 2%. In particular, we regard 10% IoU as the benchmark for severe detection fault and the OPPs corresponding to maximum average IoU within 10% are shown as green curve. Therefore, it can be concluded that our global fault injection can almost completely destroy the detection accuracy of the SkyNet hardware IP regardless of the input images.

To better illustrate our DVFS attack, we randomly select 4 images from the whole test dataset and run the accelerator with and without our DVFS fault injection. The corresponding results are shown in Fig. 6. The cyan bounding boxes represent the ground truth. The blue bounding boxes represent the results generated without DVFS regulation (i.e., *the normal truth*). And the red bounding boxes represent 100 detection results when the accelerator works under (350MHz, 692mV), one of the ten most vulnerable OPPs. The IoU of the rightmost image with the most intersection area is around 3% while the IoU of the other images are almost 0%. For the other images of the dataset, the evaluation results stay similar.

## 4.4 Local Fault Injection

The SkyNet accelerator processes four images in parallel in one single inference procedure. Therefore, we select four images (img0, img1, img2, img3) with quite different contents from the test dataset of 1000 images to show the results of the local fault injection.

As introduced in Sec.3.1, in our local fault injection setting, we first fix the injection duration, inject a single DVFS fault in each experiment round and adjust the injection starting time. The intuition is that we can dig out which layer is more vulnerable and which layer is more robust to our attack. Due to the precision of voltage scaling time, we do not consider an injection duration less than 1 ms. We set our OPP as (350 MHz, 700 mV) which is proved to be a vulnerable OPP but without a hang or crash, while the other part of the inference procedure is under vendor-stipulated OPP. As shown in Fig. 8, we set the fault injection duration to n=1 ms and the fault injection starting time to n*k in kth round until n*k reaches the ending time of the inference procedure. The red curve represents the estimated layer in the certain time. The blue curve represents the fault probabilities and the orange curve represents the faulted IoU. Some insights could be concluded from such curve. For example, for the first 11 layers of SkyNet (time < 10 ms), the fault probabilities and the faulted IoU are both high, it indicates that those layers are easier to be affected, but the influence the attack may cause is less. When we inject fault at 40 ms, the fault probability is high and the faulted IoU is low, hence the attack can induce great destruction to the performance of network.

Next, we investigate the influence of injection duration. We select two different injection durations (2 ms and 5 ms) and repeat the injection experiments. The corresponding results are shown in Fig. 9. The red curve represents the average IoU (including the pristine one and the faulted one) and the blue curve represents the fault probabilities. By comparing three subfigures in Fig. 9, we find that with the increase of the duration time, the average IoU loss and fault probabilities may not increase as our instinct. With 1 ms duration injection, an attacker can achieve up to 100.00% fault probabilities and 71.33% average IoU loss, when the duration is raised to 5 ms, however, the corresponding results become 95.56% and 56.53%. Such finding

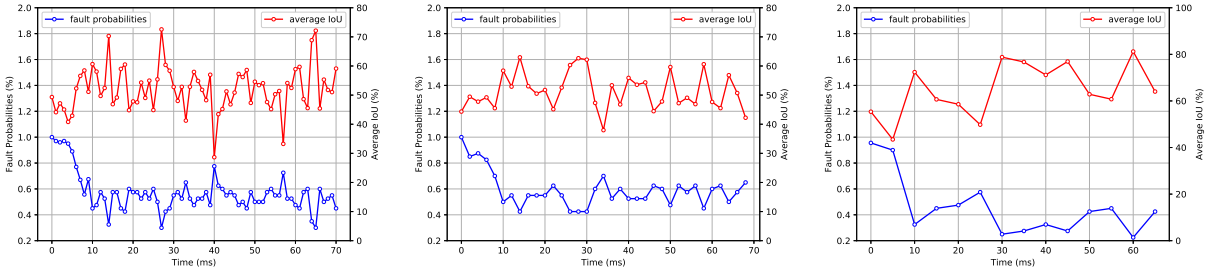


**Figure 9: Different Duration Results**

suggests that 1 ms duration is enough to induce effective SDCs and is more appropriate to inject fault in some scenarios.

Noticed that in our previous experiments, we only inject fault once in each inference procedure. Therefore, we aim to study if multi-point injection will improve our attack performance. We set the duration to 1 ms and inject the fault at both 20 ms and 40 ms of the whole procedure. The resulting average IoU loss and fault probabilities are 74.44% and 95.45%, respectively, which is higher than all the results of a single injection. Therefore, we can conclude that with multiple faults injected to the network, the attack performance can be enhanced.

## 5 CONCLUSIONS

In this paper, we propose a practical DVFS fault attack to disturb the SkyNet accelerator on FPGA by dynamically regulating the frequency/voltage pair. Through leveraging the MMCM module and the PMBus compliant power supply system, our attack can be launched remotely and stealthily without physical access to hardware. For highly encapsulated accelerator IP such as the SkyNet, we mainly demonstrate our attack in both global fault injection and local fault injection scenarios. We shows that the SkyNet accelerator on FPGAs is highly exposed to data corruption when attacker set OPPs closed to the hang-crash boundary. For global fault injection, we explore ten vulnerable OPPs that can decrease the average IoU below 2% and find two continuous OPPs lines corresponding to 90% IoU loss and the beginning of the IoU loss, respectively. For local fault injection, through rough estimation of each layer's runtime, we find out that the first 11 layers of SkyNet are easier to be affected. With 1 ms duration, our attack can achieve up to 100.00% fault probabilities and 71.33% average IoU loss. For two-point injection, the average IoU loss and fault probabilities are 74.44% and 95.45%.

## ACKNOWLEDGMENTS

This work was supported in part by National Key R&D Program of China (2020AAA0107700), by National Natural Science Foundation of China (62072398, 62032021, 61772236), by Alibaba-Zhejiang University Joint Institute of Frontier Technologies, by National Key Laboratory of Science and Technology on Information System Security, by State Key Laboratory of Mathematical Engineering and Advanced Computing, and by Key Laboratory of Cyberspace Situation Awareness of Henan Province.

## REFERENCES

[1] Michel Agoyan et al. 2010. When Clocks Fail: On Critical Paths and Clock Faults. In *Smart Card Research and Advanced Application*, Dieter Gollmann, Jean-Louis Lanet, and Julien Iguchi-Cartigny (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 182–193.

[2] Kyeongryeol Bong et al. 2017. Low-Power Convolutional Neural Network Processor for a Face-Recognition System. *IEEE Micro* 37, 6 (2017), 30–38.

[3] Jakub Breier et al. 2018. Practical Fault Attack on Deep Neural Networks. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) *(CCS '18)*. Association for Computing Machinery, New York, NY, USA, 2204–2206.

[4] Yu-Hsin Chen et al. 2017. Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks. *IEEE Journal of Solid-State Circuits* 52, 1 (2017), 127–138.

[5] Kaiyuan Guo et al. 2018. Angel-Eye: A Complete Design Flow for Mapping CNN Onto Embedded FPGA. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 37, 1 (2018), 35–47.

[6] Sanghyun Hong et al. 2019. Terminal Brain Damage: Exposing the Graceless Degradation in Deep Neural Networks Under Hardware Fault Attacks. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 497–514.

[7] Sanghyun Hong et al. 2019. Terminal Brain Damage: Exposing the Graceless Degradation in Deep Neural Networks Under Hardware Fault Attacks. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 497–514.

[8] Weixiong Jiang et al. 2020. Optimizing energy efficiency of CNN-based object detection with dynamic voltage and frequency scaling. *Journal of Semiconductors* 41, 2 (2020), 022406.

[9] Yoongu Kim et al. 2014. Flipping Bits in Memory without Accessing Them: An Experimental Study of DRAM Disturbance Errors. In *Proceeding of the 41st Annual International Symposium on Computer Architecuture* (Minneapolis, Minnesota, USA) *(ISCA '14)*. IEEE Press, 361–372.

[10] Yixing Li et al. 2017. A 7.663-TOPS 8.2-W Energy-Efficient FPGA Accelerator for Binary Convolutional Neural Networks (Abstract Only) *(FPGA '17)*. Association for Computing Machinery, New York, NY, USA, 290–291.

[11] Wenye Liu et al. 2020. Imperceptible Misclassification Attack on Deep Learning Accelerator by Glitch Injection. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. 1–6.

[12] Wenye Liu et al. 2020. Imperceptible Misclassification Attack on Deep Learning Accelerator by Glitch Injection. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. 1–6.

[13] Mohammad Motamedi et al. 2017. Machine Intelligence on Resource-Constrained IoT Devices: The Case of Thread Granularity Optimization for CNN Inference. *ACM Trans. Embed. Comput. Syst.* 16, 5s, Article 151 (Sept. 2017), 19 pages.

[14] Pengfei Qiu et al. 2019. VoltJockey: Breaking SGX by Software-Controlled Voltage-Induced Hardware Faults. In *2019 Asian Hardware Oriented Security and Trust Symposium (AsianHOST)*. 1–6.

[15] Majid Sabbagh et al. 2020. A Novel GPU Overdrive Fault Attack. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. 1–6.

[16] Giulia Santoro et al. 2018. Design-Space Exploration of Pareto-Optimal Architectures for Deep Learning with DVFS. In *2018 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5.

[17] Xiaofan Zhang et al. 2019. SkyNet: A Champion Model for DAC-SDC on Low Power Object Detection.

[18] Pu Zhao et al. 2019. Fault Sneaking Attack: a Stealthy Framework for Misleading Deep Neural Networks. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*. 1–6.